SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 삼성청년 SW·AI아카데미

## AI 코어 PJT

Confidential

# <알림>

본 강의는 삼성청년SW·AI아카데미의 컨텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로
복사, 촬영, 녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를 이용한 보관,
제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Day2. 멀티모달 인식 모델 연동

Confidential

# 챕터의 포인트

- 인식 결과를 텍스트로 (YOLO · VLM)
- MediaPipe 제스처 인식
- 웹 대시보드 서버
- LLM 상황 통합

Confidential

# 2회차 목표 : 멀티모달 인식 통합과 의도 파악 AI Workflow

- **1일차 PoC 서버(이미지 in → 텍스트 out)를 이어받아 손 제스처를 더하고, 웹 대시보드에서 LLM 답변까지 잇는다**
- **계획서 2일차 = 제스처 · VLM · YOLO 결과를 Text 로 → LLM 상황 설명 - 오늘 순서(인식 Text → 제스처 → 대시보드 → LLM 통합)가 그대로 대응**

| 차시 | 주제 | 핵심 수행 과제 | 산출물 |
|---|---|---|---|
| 1일차 (완료) | PoC 제작 + Object Detection 파인튜닝 | YOLO · VLM 선정 · 테스트, PoC 서버, YOLO Finetuning | PoC 서버, measurement.csv, best.pt |
| 2일차 (오늘) | 멀티모달 인식 모델 연동 | YOLO · VLM · MediaPipe 결과를 Text 로 → 웹 대시보드 서버 → LLM 통합 | 웹 대시보드 서버 (인식 + LLM 답변) |
| 3일차 | AI Workflow 구현 | 의도 파악 + 도구 호출(tool call) 조건 분기, 시뮬레이터 연동 | 대시보드 + 시뮬레이터 동작 데모 |

Confidential

# 학습 목표

- **① 바꾼다 - 인식 결과를 LLM 이 읽을 Text 규격으로 바꾼다**
  - YOLO 는 '이름 N개', VLM 은 '한 문장, 장애물 있음/없음' 으로 형식을 고정하면 통과
- **② 더한다 - 손 제스처를 세 번째 인식 모달리티로 추가한다**
  - 사진 한 장에서 제스처 종류와 가리키는 방향을 Text 로 뽑으면 통과
- **③ 잇는다 - 웹 대시보드 화면의 요청을 처리하는 서버 API 를 만든다**
  - 웹캠 [전송] 한 번에 YOLO 박스 · VLM 문장이 화면에 뜨면 통과
- **④ 이해시킨다 - LLM 이 세 Text 로 상황을 설명하게 한다**
  - 출처 규칙이 들어간 프롬프트로 답을 받고, 대시보드 화면에 그 답이 뜨면 통과

Confidential

# 3일차는 이 서버에 도구 호출을 더한다

- **① 웹 대시보드 서버 - 화면(제공) + API**
  - /api/yolo-detect · /api/vlm-describe · /api/llm-answer. 웹캠 질문에 답이 화면에 뜬다
- **② 인식 Text 규격 - [YOLO 객체 인식] / [VLM 장면 설명] / [손 제스처]**
  - LLM 입력의 설계 그 자체. 3일차에도 같은 블록을 쓴다
- **③ VLM 단축 기준 - 긴 변 448px**
  - 읽는 시간을 줄이는 것이 가장 큰 레버 (실측 4.4배)
- **④ LLM 통합 답변 + JSON 파서**
  - 3일차에 도구 호출(tool call) 결정을 받을 그릇

Confidential

# 1일차 가상환경에 두 가지만 더한다

- **mediapipe 패키지 1개 + 제스처 모델(.task) + LLM 가중치 - llama-cpp-python 은 1일차 0.3.34 그대로**

```
# 1일차 가상환경에 이어서 설치
.venv\Scripts\activate
pip install mediapipe==1.0.1

# 제스처 인식 모델 (약 8MB) → models\   (PowerShell 에서는 반드시 curl.exe)
cd C:\work\core_pjt
curl.exe -L -o models\gesture_recognizer.task https://storage.googleapis.com/mediapipe-models/gesture_recognizer/gesture_recognizer/float16/latest/gesture_recognizer.task

# LLM 가중치 (약 2.4GB) → models\   (1일차에 받았으면 생략)
hf download Qwen/Qwen3-4B-GGUF Qwen3-4B-Q4_K_M.gguf --local-dir models
```

전체 소스 : 설치_명령어_2일차.sh
https://gist.github.com/maketrust7/ec4131ca6f6514b928af0e9b70f6d0b0

Confidential

# models₩ 폴더에 다섯 파일이 모두 있어야 한다

- **2일차에 새로 받는 것은 아래 표의 마지막 두 줄**
- **mediapipe 는 한글 경로를 못 읽는다 - 스크립트가 파일을 바이트로 읽어 넘긴다**

| 파일 | 위치 | 크기 | 쓰는 곳 |
|---|---|---|---|
| yolo11n.pt | core_pjt₩ | 5.4 MB | 인식 Text · 대시보드 |
| Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf | models₩ | 1.9 GB | 인식 Text · 대시보드 |
| mmproj-Qwen2.5-VL-3B-Instruct-f16.gguf | models₩ | 1.3 GB | 인식 Text · 대시보드 |
| gesture_recognizer.task (신규) | models₩ | 8 MB | 제스처 · 대시보드 |
| Qwen3-4B-Q4_K_M.gguf (신규) | models₩ | 2.4 GB | LLM 통합 |

Confidential

# 2일차 신규 항목까지 한 번에 확인한다

- **① 패키지 ② 가중치 5개 ③ 입력 이미지 3장**
- **[2일차 신규] 표시가 붙은 줄이 오늘 새로 필요한 것**
- **실행 : python 환경체크_설치확인.py**

```python
REQUIRED_PACKAGES = [
    ("fastapi", "웹 대시보드 서버", False),
    ("uvicorn", "웹 대시보드 서버 실행", False),
    ("httpx", "3일차 API 검증", False),
    ("PIL", "이미지 처리 (pillow)", False),
    ("ultralytics", "YOLO Object Detection", False),
    ("llama_cpp", "VLM / LLM GGUF 추론", False),
    ("mediapipe", "손 제스처 인식", True),
]

# ...

WEIGHTS = [
    (PROJECT_ROOT / "yolo11n.pt", "YOLO11n", False),
    (MODELS_DIR / "Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf", "VLM 본체", False),
    (MODELS_DIR / "mmproj-Qwen2.5-VL-3B-Instruct-f16.gguf", "VLM mmproj", False),
    (MODELS_DIR / "gesture_recognizer.task", "제스처 모델", True),
    (MODELS_DIR / "Qwen3-4B-Q4_K_M.gguf", "LLM", True),
]
```

전체 소스 : page10_환경체크/환경체크_설치확인.py
https://gist.github.com/maketrust7/7bc387b620177061b5d65d3183644119

Confidential

# MISS 가 하나라도 보이면 실습을 시작하지 않는다

- **실측 : 15개 항목 모두 OK**
- **llama_cpp 는 0.3.34 인지 확인**

```
명령 프롬프트

Python 3.12.4 / Windows

[1] 패키지 설치 확인
    OK   fastapi      0.139.2    웹 대시보드 서버
    OK   uvicorn      0.51.0     웹 대시보드 서버 실행
    OK   httpx        0.27.0     3일차 API 검증
    OK   PIL          12.3.0     이미지 처리 (pillow)
    OK   ultralytics  8.4.104    YOLO Object Detection
    OK   llama_cpp    0.3.34     VLM / LLM GGUF 추론
    OK   mediapipe    1.0.1      손 제스처 인식 [2일차 신규]

[2] 모델 가중치 확인
    OK   YOLO11n          5.4 MB  yolo11n.pt
    OK   VLM 본체       1840.5 MB  Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf
    OK   VLM mmproj    1276.4 MB  mmproj-Qwen2.5-VL-3B-Instruct-f16.gguf
    OK   제스처 모델          8.0 MB  gesture_recognizer.task [2일차 신규]
    OK   LLM           2381.6 MB  Qwen3-4B-Q4_K_M.gguf [2일차 신규]

[3] 입력 이미지 확인
    OK   samples/sample.jpg
    OK   samples/pointing_up.jpg
    OK   samples/victory.jpg

------------------------------------------------------------
모든 준비 완료 — 실습을 시작하세요.
```

# 인식 결과를 텍스트로

Confidential

# LLM 은 이미지를 보지 않는다 - 인식 모델이 만든 몇 줄의 Text 가 전부다

- **형식이 흔들리면 ('person 4개' ↔ '사람 네 명') 같은 규칙이 매번 다르게 먹힌다**
- **출처가 섞이면 누구를 믿을지 못 정한다 - 줄 앞에 [YOLO 객체 인식] 같은 출처 표시를 붙인다**
- **출처 표시는 55쪽 근거 충돌 규칙에 그대로 쓰인다**
- **그래서 세 인식 결과의 형식을 오늘 고정한다 - 바꾸면 LLM 프롬프트도 같이 고쳐야 한다**

| 출처 | 형식 | 예시 (실측) | 쪽 |
|---|---|---|---|
| [YOLO 객체 인식] | 이름 N개, 이름 M개 (많은 순) | person 4개, bus 1개 | 14쪽 |
| [VLM 장면 설명] | 한국어 한 문장, 40자 이내, 끝에 '장애물 있음/없음' | 버스 앞에 사람들이 걷고 있다, 장애물 없음. | 18쪽 |
| [손 제스처] | 카테고리, 가리키는 방향: 위/아래/왼쪽/오른쪽 | Pointing_Up, 가리키는 방향: 위 | 31쪽 |

Confidential

# 좌표를 버리고 '무엇이 몇 개' 만 남긴다

- **Counter 로 클래스별 개수를 센다**
- **most_common() 으로 많은 순 정렬 - 순서도 형식이다**
- **실행 : python YOLO결과_텍스트요약.py**

```python
def summarize(model, results, min_conf: float = 0.0) -> str:
    counter = Counter(
        model.names[int(box.cls)]
        for box in results[0].boxes
        if float(box.conf) >= min_conf
    )
    if not counter:
        return NO_OBJECT_TEXT
    return ", ".join(f"{name} {count}개" for name, count in counter.most_common())
```

전체 소스 : page14_YOLO_텍스트요약/YOLO결과_텍스트요약.py
https://gist.github.com/maketrust7/7493b0c9c2ad0a65207ef669017eeb9c

Confidential

# 박스 5개가 한 문장이 된다

- ① 좌표 · conf 를 버리고 이름만 남긴다 → bus, person ×4
- ② 이름별로 센다 (Counter) → person 4, bus 1
- ③ 많은 순으로 '이름 N개' 를 쉼표로 잇는다 → person 4개, bus 1개
- min_conf 0.8 에서는 person 이 3개로 줄어든다

명령 프롬프트

```
입력 이미지 : sample.jpg

[원본 검출 결과]
  bus          conf=0.94
  person       conf=0.89
  person       conf=0.88
  person       conf=0.86
  person       conf=0.62

[LLM 입력용 요약 문장]
  [YOLO 객체 인식] person 4개, bus 1개

[min_conf 별 요약 비교 — 노이즈 ↔ 누락]
  min_conf=0.0 → person 4개, bus 1개
  min_conf=0.5 → person 4개, bus 1개
  min_conf=0.8 → person 3개, bus 1개

이 한 문장이 VLM 설명문과 함께 LLM 입력으로 들어갑니다.
```

Confidential

# 임계값은 두 번 건다 : 검출할 때(conf), 요약할 때(min_conf)

- **conf - YOLO 가 박스를 낼지 말지 (검출 단계)**
  - 1일차 실습 2-1 에서 본 값. 기본 0.25
- **min_conf - 낸 박스 중 무엇을 Text 에 넣을지 (요약 단계)**
  - 실측 : 0.5 까지는 person 4개, 0.8 에서 person 3개 (0.62 짜리가 빠짐)
- **로봇 입장에서 둘은 비용이 다르다**
  - 노이즈(없는 것을 봄) → 괜히 멈춘다.  누락(있는 것을 못 봄) → 부딪힌다
  - 장애물 판단용이면 누락이 더 위험하다 - 낮게 잡고 LLM 에 개수를 맡긴다

Confidential

# VLM 에게는 '서술' 만 시키고, 형식은 문장으로 못 박는다

- **max_tokens - 출력 길이의 상한을 건다 (40자는 프롬프트로 요청)**
  - 60 토큰 ≈ 한국어 약 90자 - 상한일 뿐, 40자 이내를 보장하지는 않는다
- **언어 · 문장 수 - 설정 옵션이 없어 프롬프트에 적는다 ("한국어 한 문장으로")**
  - 1일차 실측 : 적지 않으면 영어 279자가 나왔다
- **장애물 유무 - 문장 끝에 '장애물 있음/없음' 을 붙이게 한다**
  - '포함해 줘' 로 부탁하면 두 번째 문장으로 따로 쓴다 → 한 문장 규격이 깨진다
  - 형식 예시를 한 줄 주면 따라 한다. 단, 장면과 무관한 예시를 준다

Confidential

# 규격은 프롬프트/max_tokens/코드 세 군데에 건다

- 입력은 긴 변 448px 로 줄인다
- 이유 : 810x1080 → 336x448 (픽셀 1/6, 약 4배 빠름)
- normalize 는 다듬기, check 는 검사
- 실행 : python VLM_출력규격.py (1회 약 8초)

```python
SPEC_PROMPT = (
    "이 이미지를 한국어 한 문장으로 설명해줘. 40자 이내로 쓰고, 문장 끝에 "
    "'장애물 있음' 또는 '장애물 없음' 을 붙여줘. "
    "형식 예: 복도에 의자 두 개가 놓여 있다, 장애물 있음."
)
SPEC_MAX_CHARS = 40
SPEC_MAX_TOKENS = 60

# ...

        max_tokens=SPEC_MAX_TOKENS,    # 길이는 프롬프트 + max_tokens 양쪽에서 건다
        temperature=0.0,               # 재현성

# ...

def normalize(text: str) -> str:
    text = " ".join(text.split()).lstrip(":. ")
    first = re.split(r"(?<=[.!?。])\s", text, maxsplit=1)[0].strip()
    obstacle = re.search(r"장애물 (있음|없음)", text)
    if obstacle and "장애물" not in first:
        first = f"{first.rstrip(' .')}, {obstacle.group(0)}."
    return first
```

전체 소스 : page18_VLM_출력규격/VLM_출력규격.py
https://gist.github.com/maketrust7/05417d134db47f5fe1dd42d4657bff29

Confidential

# 두 번 돌려도 같은 문장, 규격 검사 통과

- **448px 입력 : 1회 7~8초**
- **원문 앞의 ':' 는 normalize 가 지운다**
- **재현성 : 2회 모두 같은 문장**
- **다른 이미지 : python VLM_출력규격.py --image C:₩work₩core_pjt₩source₩2회차_day2₩samples₩victory.jpg --repeat 1 → 딱 40자 (경계)**

```
명령 프롬프트

입력 이미지 : sample.jpg → 336x448
프롬프트    : 이 이미지를 한국어 한 문장으로 설명해줘. 40자 이내로 쓰고, 문장
              끝에 '장애물 있음' 또는 '장애물 없음' 을
              붙여줘. 형식 예: 복도에 의자 두 개가 놓여
              있다, 장애물 있음.
max_tokens  : 60 / temperature : 0.0

[1회차] 7.9s
  원문      : : 버스 앞에 사람들이 걷고 있다, 장애물 없음.
  규격화    : 버스 앞에 사람들이 걷고 있다, 장애물 없음.  (25자)
  검사      : 한국어 OK / 40자 이내 OK / 장애물 표기 OK
[2회차] 6.8s
  원문      : : 버스 앞에 사람들이 걷고 있다, 장애물 없음.
  규격화    : 버스 앞에 사람들이 걷고 있다, 장애물 없음.  (25자)
  검사      : 한국어 OK / 40자 이내 OK / 장애물 표기 OK

재현성 : 같은 문장 (2회)

이 한 문장이 [VLM 장면 설명] 블록으로 LLM 에 들어갑니다.
```

Confidential

# 모델을 믿지 말고, 형식은 코드로 확인한다

- **재현성 - temperature=0.0**
  - 같은 이미지에 같은 문장. 없으면 프롬프트를 비교할 수조차 없다
- **normalize - 코드로 다듬는 것**
  - 앞에 붙는 '::' 같은 불필요한 문자, 줄바꿈 제거. 두 문장이면 첫 문장만
- **check - 코드로 검사만 하는 것**
  - 40자를 넘겼다고 자르면 끝의 '장애물 없음' 이 잘린다 → 자르지 말고 기록한다
- **입력 축소 - 긴 변 448px**
  - 원본 해상도에서는 한자 · 두 문장이 섞였다 (1일차 도전 2). 줄이면 규격도 지키고 약 4배 빠르다
  - 실측 : victory.jpg 는 딱 40자(경계). 새로 설치한 PC 에서는 '도로에停해' → 한국어 NG

# 도전 문제 1 - 요약 함수 완성하기

## YOLO결과_텍스트요약_도전1.py 의 summarize() 를 아래 규칙에 맞게 고치시오.

### 요구 사항

① conf 0.5 미만 검출은 버린다
② 출력 형식을 다음 한 줄로 고정한다
    총 5개 (person 4개, bus 1개)
③ 많은 순, 개수가 같으면 이름 순
    car 2개, person 2개
④ tie 같은 소지품(NOT_OBSTACLE)은 빼고 센다

### 힌트

· ① 은 min_conf 기본값만 바꿔도 된다
· 총 개수는 Counter 값의 합
· most_common() 은 동률이면 먼저 센 순
  → sorted() 에 key 를 준다
· 실행하면 사례마다 O / X 가 찍힌다
  (③ ④ 는 가상 검출로 확인)

전체 소스 : page21_YOLO_텍스트요약/YOLO결과_텍스트요약_도전1.py
https://gist.github.com/maketrust7/7bf0e5918851a5da51651838a42830b7

# 도전 1 해설 - 형식은 함수 하나에 가둔다

Confidential

## 형식을 한 곳에서만 만들면, 바꿀 때도 한 곳만 고친다

- ① min_conf 기본값 0.5
- ② 총합 = sum(counter.values())
- ③ key=(-개수, 이름) 로 정렬
- ④ 거를 때 NOT_OBSTACLE 도 뺀다

```python
def summarize(model, results, min_conf: float = 0.5) -> str:
    counter = Counter(
        model.names[int(box.cls)]
        for box in results[0].boxes
        if float(box.conf) >= min_conf                    # ① 0.5 미만은 버린다
        and model.names[int(box.cls)] not in NOT_OBSTACLE   # ④ 소지품 제외
    )
    if not counter:
        return NO_OBJECT_TEXT
    # ③ 많은 순, 개수가 같으면 이름 순
    items = sorted(counter.items(), key=lambda kv: (-kv[1], kv[0]))
    text = ", ".join(f"{name} {count}개" for name, count in items)
    return f"총 {sum(counter.values())}개 ({text})"          # ② 형식 고정
```

검증 – 정답 파일 실제 실행

```
[도전 1] summarize() 채점 (conf 0.25 로 검출한 뒤 요약)
  O  sample.jpg              → 총 5개 (person 4개, bus 1개)
  O  pointing_up.jpg         → 총 1개 (person 1개)
  O  victory.jpg             → 인식된 객체 없음
  O  가상 검출 (동률 · tie 0.90)    → 총 5개 (car 2개, person 2개, bus 1개)

  4/4 통과
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# MediaPipe 제스처 인식

Confidential

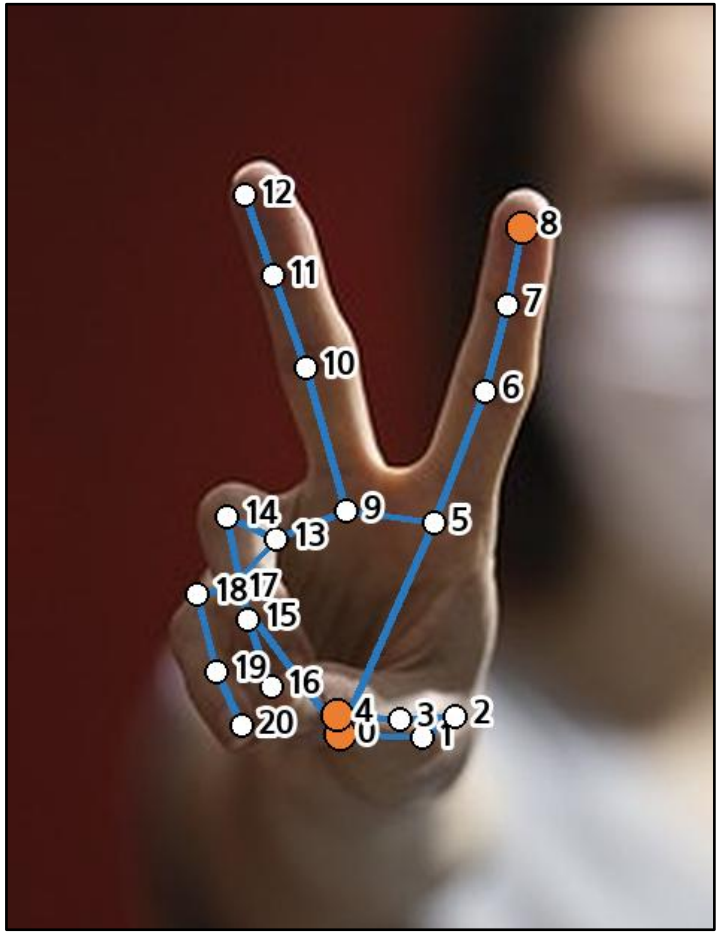
# 말로 하기 애매한 것을 손으로 전달한다

- **MediaPipe - Google 의 온디바이스 비전 솔루션 모음**
  - 손 · 얼굴 · 포즈 · 객체 등. 가벼운 모델이라 CPU 에서도 실시간
  - Python 패키지 하나(pip install mediapipe)와 모델 파일(.task) 하나로 동작
- **왜 제스처인가**
  - '저쪽으로 가' 의 '저쪽' 은 말보다 손이 정확하다
  - 카메라 한 대로 장면(YOLO · VLM)과 사용자의 의도(손)를 함께 본다
- **오늘의 역할 - 방향을 Text 로**
  - '가리키는 방향: 위' 한 줄을 만들어 LLM 입력에 더한다

Confidential

# 번호는 손목 0 에서 시작해 손가락마다 4점씩

- 0 : 손목
- 1~4 엄지 / 5~8 검지 / 9~12 중지
- 13~16 약지 / 17~20 새끼
- 각 손가락 끝 = 4 · 8 · 12 · 16 · 20
- 좌표는 0~1 정규화, 원점은 좌상단
- 주황색 = 오늘 쓰는 0 · 4 · 8



Confidential

# 손을 찾고 → 관절 21점을 찍고 → 모양을 7종 + None 중 하나로 분류한다

- **한 번 호출에 셋이 함께 나온다 : hand_landmarks · gestures · handedness(왼/오른손)**
- **학습하지 않은 모양은 None 이 되거나 엉뚱한 이름이 된다 - 오늘은 랜드마크를 주로 쓴다**
- **방향(왼쪽/오른쪽)은 카테고리에 없다 → 랜드마크로 직접 계산한다 (실습 2-2)**

| 카테고리 | 손 모양 | 카테고리 | 손 모양 |
|---|---|---|---|
| Closed_Fist | 주먹 | Thumb_Down | 엄지 아래 |
| Open_Palm | 편 손바닥 | Thumb_Up | 엄지 위 |
| Pointing_Up | 검지로 위 가리키기 | Victory | V 사인 |
| ILoveYou | 엄지 · 검지 · 새끼 펴기 | None | 해당 없음 |

Confidential

# 이미지 1장 → 제스처 · 왼/오른손 · 랜드마크 21점

- **모델 파일은 경로 대신 바이트로 넘긴다**
- **이미지는 PIL → numpy → mp.Image**
- **실행 : python 제스처_인식.py --save**

경로(model_asset_path)로 주면 mediapipe 가 파일을 직접 연다
→ 경로에 한글이 있으면 열지 못한다
→ 파이썬이 read_bytes() 로 읽은 내용만 넘긴다

```
GESTURE_MODEL_PATH = PROJECT_ROOT / "models" / "gesture_recognizer.task"

base_options=mp_tasks.BaseOptions(
    model_asset_buffer=GESTURE_MODEL_PATH.read_bytes()),
```

```
    options = vision.GestureRecognizerOptions(
        # 경로 대신 바이트로 넘긴다 (한글 경로 문제 회피)
        base_options=mp_tasks.BaseOptions(
            model_asset_buffer=GESTURE_MODEL_PATH.read_bytes()),
        running_mode=vision.RunningMode.IMAGE,
        num_hands=1,
    )
    return vision.GestureRecognizer.create_from_options(options)

# ...

def to_mp_image(image: Image.Image):
    import mediapipe as mp

    return mp.Image(image_format=mp.ImageFormat.SRGB, data=np.asarray(image))

# ...

        result = recognizer.recognize(to_mp_image(image))

# ...

        gesture = result.gestures[0][0]
        hand = result.handedness[0][0]
        landmarks = result.hand_landmarks[0]
```

전체 소스 : page27_제스처_인식/제스처_인식.py
https://gist.github.com/maketrust7/97223a3daeb7dd624bd9929087c321c6

Confidential

# 두 장 모두 손을 찾고 모양을 맞혔다

- **실측 : Pointing_Up 0.82 / Victory 0.91**
- **--save 로 21점을 그린 PNG 가 폴더에 생긴다**
- **handedness 는 거울 화면 기준**

명령 프롬프트

```
입력 이미지 : pointing_up.jpg (419x419)
  gesture    : Pointing_Up (score 0.82)
  handedness : Left
  landmarks  : 21점
    [ 0] 손목     x=0.80 y=0.32
    [ 4] 엄지 끝  x=0.75 y=0.19
    [ 8] 검지 끝  x=0.71 y=0.07
  저장       : landmarks_pointing_up.png

입력 이미지 : victory.jpg (418x418)
  gesture    : Victory (score 0.91)
  handedness : Right
  landmarks  : 21점
    [ 0] 손목     x=0.29 y=0.75
    [ 4] 엄지 끝  x=0.29 y=0.74
    [ 8] 검지 끝  x=0.42 y=0.40
  저장       : landmarks_victory.png

gesture 는 '무슨 손 모양인가', landmarks 는 '손이 어디를 향하는가' 를 알려준다.
```

Confidential

# 손목(0) → 검지 끝(8) 벡터가 가리키는 쪽이 방향이다

- **dx, dy 를 구한다**
  - dx = 검지 끝 x − 손목 x,  dy = 검지 끝 y − 손목 y
- **더 큰 쪽이 축을 정한다**
  - |dx| > |dy| 이면 좌우, 아니면 상하
- **부호가 방향을 정한다**
  - dx > 0 → 오른쪽,  dy > 0 → 아래 (y 는 아래로 커진다)
- **너무 짧으면 판단을 보류한다**
  - 주먹처럼 손가락이 접히면 벡터가 짧다 → '인식 불가' 가 오답보다 낫다

# 방향 판별 원리 ② - 카테고리 vs 랜드마크

Confidential

## 같은 손을 돌려 보면 카테고리는 틀리고 랜드마크는 맞는다



**0° 회전**
Pointing_Up (0.82)
랜드마크 → 위



**90° 회전**
None (0.78)
랜드마크 → 왼쪽



**180° 회전**
Thumb_Down (0.48)
랜드마크 → 아래



**270° 회전**
None (0.75)
랜드마크 → 오른쪽

- **정규화 좌표(0~1)는 픽셀로 되돌린 뒤 비교한다 - 가로 · 세로 비율이 다를 수 있다**
- **카테고리는 '세운 손' 위주로 학습돼서, 방향이 돌아간 손은 None 이나 엉뚱한 이름이 된다**

Confidential

# 벡터 4줄 + 분기 4줄이면 방향이 나온다

- **rotate() 로 한 장을 네 방향으로 만든다**
- **to_text() 가 LLM 입력 형식을 고정한다**
- **실행 : python 제스처_방향판별.py --save**

```python
for angle in (0, 90, 180, 270):
    image = base.rotate(angle, expand=True)   # 반시계 방향 회전
```



**0° 회전**
Pointing_Up (0.82)
랜드마크 → 위



**90° 회전**
None (0.78)
랜드마크 → 왼쪽



**180° 회전**
Thumb_Down (0.48)
랜드마크 → 아래



**270° 회전**
None (0.75)
랜드마크 → 오른쪽

```python
def classify_direction(landmarks, width: int, height: int,
                       mirror: bool = False) -> str:
    wrist, tip = landmarks[WRIST], landmarks[INDEX_TIP]
    dx = (tip.x - wrist.x) * width
    dy = (tip.y - wrist.y) * height          # y 는 아래로 커진다
    if math.hypot(dx, dy) < min(width, height) * MIN_VECTOR_RATIO:
        return "unknown"
    if abs(dx) > abs(dy):
        is_right = dx > 0
        if mirror:                           # 거울 화면이면 좌우만 뒤집는다
            is_right = not is_right
        return "right" if is_right else "left"
    return "down" if dy > 0 else "up"

# ...

def to_text(category: str, direction: str) -> str:
    return f"손 제스처: {category}, 가리키는 방향: {DIRECTION_KO[direction]}"
```

전체 소스 : page31_제스처_방향판별/제스처_방향판별.py
https://gist.github.com/maketrust7/f7610167848d8459d4fadb0846ecaaba

Confidential

# 카테고리는 네 번 중 세 번 틀리고, 방향은 네 번 모두 맞다

- **90° · 270° 는 None, 180° 는 Thumb_Down**
- **랜드마크 방향 : 위 → 왼쪽 → 아래 → 오른쪽**
- **→ 방향은 카테고리가 아니라 랜드마크로**

명령 프롬프트

```
입력 이미지 : pointing_up.jpg   mirror=False

   회전 | gesture 카테고리           | 랜드마크 방향
-------------------------------------------------
   0° | Pointing_Up (0.82)     | 위
  90° | None (0.78)            | 왼쪽
 180° | Thumb_Down (0.48)      | 아래
 270° | None (0.75)            | 오른쪽

[LLM 입력용 Text]
  손 제스처: Pointing_Up, 가리키는 방향: 위
  손 제스처: None, 가리키는 방향: 왼쪽
  손 제스처: Thumb_Down, 가리키는 방향: 아래
  손 제스처: None, 가리키는 방향: 오른쪽

저장 : direction_pointing_up.png

카테고리는 손이 돌아가면 틀린다 → 방향은 랜드마크로 판단한다.
```

Confidential

# 화면의 왼쪽이 사용자의 왼쪽이 아닐 수 있다

- **셀카 카메라(노트북 웹캠)는 보통 좌우를 뒤집어 보여준다**
  - 사용자가 자기 오른쪽을 가리키면 원본 이미지에서는 왼쪽을 가리킨 것이 된다
  - 해결 : --mirror 처럼 좌우 판정만 뒤집는다. 위 · 아래는 그대로
  - 실측 : --mirror 를 주면 90° 가 오른쪽, 270° 가 왼쪽으로 바뀐다
- **handedness 도 거울 화면을 가정한다**
  - pointing_up.jpg 는 사진 속 사람의 왼손인데 'Left' 로 나온다 - 우연히 맞은 것
  - 뒤집지 않은 카메라(라즈베리파이 등)에서는 반대로 나올 수 있다
- **카메라마다 한 번씩 실제 손으로 확인하고 설정을 고정한다**

Confidential

# 한 줄 형식 + '모름' 을 표현하는 방법까지 정한다

- **형식 - 카테고리, 가리키는 방향: 위/아래/왼쪽/오른쪽**
  - 예 : Pointing_Up, 가리키는 방향: 위
- **손이 없을 때 - '손 없음'**
  - 빈칸으로 두면 LLM 은 손이 없는 건지, 인식이 실패한 건지 알 수 없다
  - 그러면 없는 내용을 짐작해 답에 채워 넣을 수 있다 - '손 없음' 이라고 적어 추측할 여지를 없앤다
- **방향을 모를 때 - '인식 불가'**
  - 짐작한 방향보다 모른다는 표시가 안전하다 - 로봇이 엉뚱한 쪽으로 가지 않는다
  - 대시보드는 방향만 보내므로 둘을 합쳐 '인식 불가' 한 가지로 쓴다
- **카테고리는 참고용으로만 남긴다**
  - LLM 규칙에서 방향의 근거는 '가리키는 방향' 쪽이다

Confidential

# 실습 중 막히면 여기부터 확인한다

| 증상 | 원인 | 해결 |
|---|---|---|
| FileNotFoundError: Can't find file | mediapipe 가 한글 경로를 못 읽음 | 파일을 바이트/배열로 읽어 넘긴다 (실습 코드 방식) |
| W0000 / I0000 로그가 잔뜩 나옴 | mediapipe 내부 로그 | 오류가 아님. 무시 |
| result.gestures 가 빈 리스트 | 손을 못 찾음 (작거나 가려짐) | '손 없음' 으로 처리. 손을 화면 가운데로 |
| 방향이 좌우 반대로 나옴 | 셀카 카메라의 거울 화면 | --mirror (좌우만 뒤집기) |
| 카테고리가 엉뚱함 | 방향이 돌아간 손 / 학습 안 된 모양 | 방향은 랜드마크로 판단 |

# 도전 문제 2 - 손가락 개수 세기

제스처_인식_도전2.py 의 count_fingers() 를 작성해 펼친 손가락 개수(0~5)를 세시오.

### 요구 사항

① 입력은 result.hand_landmarks[0] (21점)
② victory.jpg 는 2, pointing_up.jpg 는 1
③ 손을 90° 돌려도 같은 값이 나와야 한다
④ 3일차 대시보드의 '손가락 = 칸 수' 입력과 같은 원리

### 힌트

· 손가락 끝 : 4 · 8 · 12 · 16 · 20
· 중간 관절(PIP) : 6 · 10 · 14 · 18
· '끝의 y 가 관절보다 위' 는 ③ 에서 깨진다
· 회전과 무관한 기준점은? → 손목(0)
· 엄지는 옆으로 펴진다 - 따로 생각할 것

전체 소스 : page36_제스처_인식/제스처_인식_도전2.py
https://gist.github.com/maketrust7/27c15a45e50fbd36ed6a13e85b52a3e1

Confidential

# 손끝이 관절보다 손목에서 더 멀면 펴진 손가락이다

- y 비교는 손이 돌면 깨진다
- 거리 비교는 회전과 무관하다
- 엄지는 손목 기준이 안 통한다
- → 새끼 뿌리(17)에서 멀어지면 펴짐
- 실측 : 0° · 90° 모두 같은 값

```python
TIPS = [8, 12, 16, 20]      # 검지 ~ 새끼 끝
PIPS = [6, 10, 14, 18]      # 검지 ~ 새끼 중간 관절


def dist(a, b):
    return math.hypot(a.x - b.x, a.y - b.y)


def count_fingers(landmarks) -> int:
    wrist = landmarks[0]
    # 네 손가락 : 끝이 중간 관절보다 손목에서 멀면 펴진 것
    count = sum(dist(landmarks[t], wrist) > dist(landmarks[p], wrist)
                for t, p in zip(TIPS, PIPS))
    # 엄지 : 옆으로 펴지므로 새끼 뿌리(17)에서 멀어지는지로 본다
    pinky_mcp = landmarks[17]
    if dist(landmarks[4], pinky_mcp) > dist(landmarks[3], pinky_mcp):
        count += 1
    return count
```

검증 – 정답 파일 실제 실행

```
[도전 2] count_fingers() 채점 (0° 와 90° 로 돌려 비교)
  O  pointing_up.jpg    0° → 펼친 손가락 1개  (기대 1개, gesture: Pointing_Up)
  O  pointing_up.jpg   90° → 펼친 손가락 1개  (기대 1개, gesture: None)
  O  victory.jpg        0° → 펼친 손가락 2개  (기대 2개, gesture: Victory)
  O  victory.jpg       90° → 펼친 손가락 2개  (기대 2개, gesture: None)

  4/4 통과
```

Confidential

# 세 인식 결과가 LLM 이 읽을 Text 한 줄씩이 되었다

- ① YOLO - 좌표를 버리고 '이름 N개' 만 남긴다. min_conf 는 노이즈와 누락의 교환
- ② VLM - 한국어 한 문장 + 장애물 있음/없음. 프롬프트로 요청하고 코드로 검사한다
- ③ VLM 입력은 긴 변 448px 로 줄인다 - 약 4배 빠르다
- ④ 제스처 - 카테고리는 참고만, 방향은 손목 → 검지 끝 벡터로 구한다
- ⑤ '모름' 도 형식으로 적는다 - 손 없음 · 인식 불가
- ⑥ 다음 : 브라우저가 이 세 줄을 모아 서버에 보내고, 서버가 블록으로 묶어 LLM 입력으로 넣는다

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 웹 대시보드 서버

Confidential

# 서버를 만들기 전에 가장 느린 것부터 줄인다

- **1일차 도전 3 : 두 엔드포인트를 병렬로 불러도 이득 0%**
- **병렬화의 이득은 '두 번째로 느린 작업' 만큼뿐이다**
- **그래서 순서를 바꾼다 : ① VLM 단축 → ② 대시보드 서버**

| | YOLO | VLM | 순차 합계 | 병렬 (실측) |
|---|---|---|---|---|
| 1일차 PoC (CPU) | 87 ms | 92,478 ms | 92,565 ms | 92,850 ms |
| 비중 | 0.1 % | 99.9 % | | 이득 없음 |

Confidential

# VLM 시간 = 이미지를 읽는 시간 + 답을 쓰는 시간

- **출력 토큰을 1/4 로 줄여도 2.9초, 이미지를 448px 로 줄이자 4.4배 빨라졌다**
- **대시보드 서버는 입력 줄이기를 쓴다 - 시간은 PC마다, 특히 GPU 유무에 따라 크게 다르다**

| 설정 | 바꾼 것 | 이미지 | 출력 토큰 | 시간 (실측) |
|---|---|---|---|---|
| 기준 | — | 810x1080 | 61 | 35.7 s |
| 출력 줄이기 | max_tokens 60 + 규격 프롬프트 | 810x1080 | 15 | 32.8 s |
| 입력 줄이기 | 출력 줄이기 + 긴 변 448px 로 축소 | 336x448 | 16 | 7.4 s |
| 잃는 것 | 작은 물체 · 글자를 놓친다 / 설명이 짧아진다 | | | |

Confidential

# 프런트는 완성본, 우리가 만드는 것은 브라우저의 요청을 처리하는 서버 API

- **프런트(index.html)는 3일차까지 쓰는 완성본이다 - source₩2회차_web**
  - 오늘 서버의 응답에는 tool 이 없다 → 화면은 답변만 답변창에 띄우고 로봇은 움직이지 않는다
  - 그래서 화면 아래 진행 표시의 '5.이동중' 단계도 건너뛴다
- **우리가 만드는 것은 서버 - 오늘 : YOLO + VLM (45쪽) → LLM 답변 (64쪽)**
- **제스처는 브라우저에서 돈다 - 서버는 MediaPipe 모델 파일만 내준다**
  - 방향은 실습 2-2 와 같은 손목 → 검지 끝 벡터로 계산한다
  - 주의 : 주먹을 쥐면 '제스처 입력 모드'(주먹 → 방향 → 손가락 칸 수 → 손 빼면 전송)가 켜진다
- **접속은 서버 주소로 - http://127.0.0.1:8000**
  - html 파일을 더블클릭해서 열면 API 호출 · 웹캠 · MediaPipe 로딩이 모두 실패한다

Confidential

# [전송] 한 번 → 스냅샷 1장 → YOLO · VLM 동시 요청 → LLM 요청

- **box 는 보낸 이미지의 픽셀 좌표 - 화면이 그 좌표로 스냅샷 위에 박스를 그린다**
- **오류는 {"error": "코드"} 로 약속한다 - 화면은 코드만 보고 안내 문구를 띄운다**

| 경로 | 요청 | 응답 | 쪽 |
|---|---|---|---|
| GET / | — | index.html | 45쪽 |
| GET /mediapipe/... | — | 제스처 인식 파일 (JS · wasm · .task) | 45쪽 |
| POST /api/yolo-detect | JPEG 바이트 | {objects: [{label, confidence, box}]} | 45쪽 |
| POST /api/vlm-describe | JPEG 바이트 | {description} | 45쪽 |
| POST /api/llm-answer | JSON (질문 + 인식 결과) | {need_more_info, message, user_content} | 64쪽 |

Confidential

# YOLO 와 VLM 은 '동시에 시작' 하고 '각자 끝나는 대로' 화면에 뜬다

- **브라우저가 YOLO 와 VLM 을 동시에 요청한다**
  - 한쪽 응답을 기다리지 않고 둘 다 보낸다. 먼저 온 결과부터 화면에 그린다
- **서버는 run_in_threadpool 로 추론을 스레드로 보낸다**
  - 1일차 PoC 통합 : 이벤트 루프에서 직접 돌리면 서버 전체가 멈춘다
  - llama.cpp · torch 는 C++ 안에서 GIL 을 놓는다 → 스레드로도 실제로 겹쳐 돈다
- **모델마다 지연 로딩 + 락 - 1일차에 검증한 그 패턴**
- **공짜는 아니다 - VLM 이 모든 코어를 쓰고 있어 같이 돌면 서로 느려진다**

Confidential

# 화면 하나, API 네 개, 모델 둘

- 인식 함수는 앞에서 정한 Text 규격 그대로
- YOLO 는 conf 0.25 로 검출한 박스를 모두 센다 - 실습 1-1 의 min_conf 는 두지 않는다 (누락이 더 위험)
- 추론은 run_in_threadpool 로
- 실행 : python 대시보드_서버.py
- 접속 : http://127.0.0.1:8000
- 확인 : 로컬 PC 탭 → '앞에 보이는 이미지 설명해' → [전송]

```python
@app.post("/api/yolo-detect")
async def yolo_detect(request: Request):
    image, error = await _read_image(request)
    if error:
        return error
    try:
        objects = await run_in_threadpool(detect_objects, image)
    except Exception as exc:          # 부분 실패 — VLM 요청은 따로 진행된다
        return JSONResponse({"error": "yolo_failed"}, status_code=500)
    return {"objects": objects}

# ...

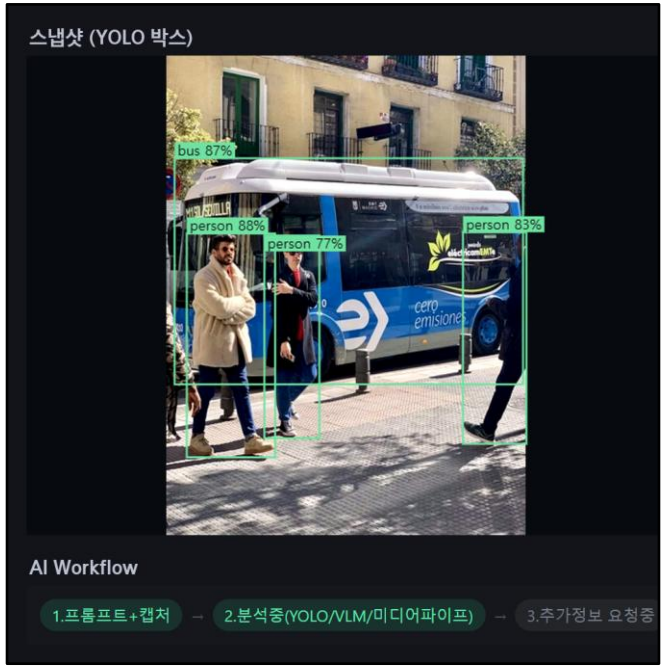
def detect_objects(image: Image.Image) -> list:
    model = _get("yolo", _load_yolo)
    with _run_locks["yolo"]:
        results = model.predict(image, conf=0.25, verbose=False)
    # box : 보낸 이미지의 픽셀 좌표 [x1, y1, x2, y2] — 화면이 그대로 그린다
    return [
        {
            "label": model.names[int(box.cls)],
            "confidence": round(float(box.conf), 3),
            "box": [round(v, 1) for v in box.xyxy[0].tolist()],
        }
        for box in results[0].boxes
    ]
```

전체 소스 : page45_대시보드_서버/대시보드_서버.py
https://gist.github.com/maketrust7/394b11a8e7e84bb752778894e0d75821

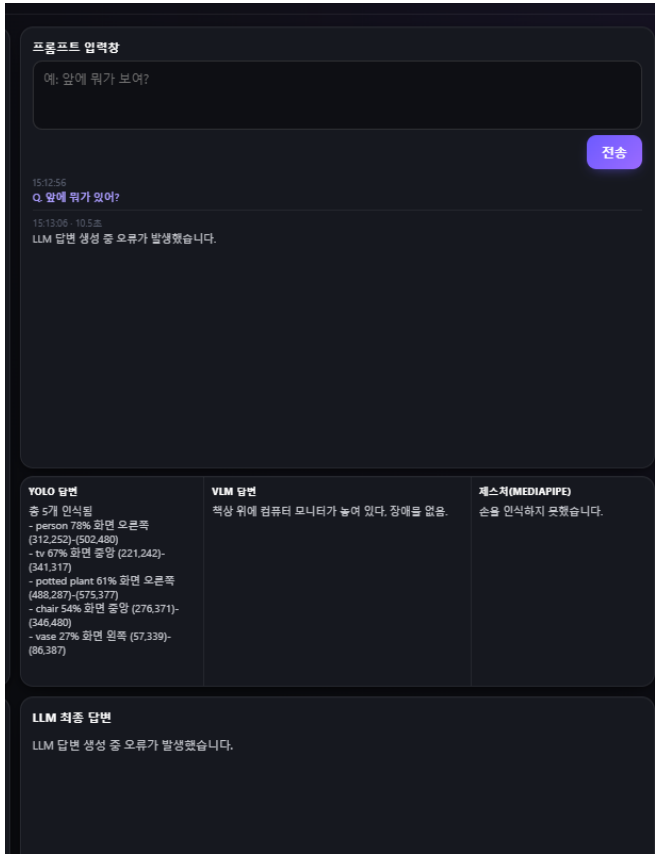
Confidential

# 웹캠 한 장이 박스와 문장이 된다

- YOLO : person 3개 · bus 1개 박스
- 실습 1-1 은 4개 - 640x480 스냅샷이라 먼 사람을 놓침
- VLM : 장면 한 문장 + 장애물 없음



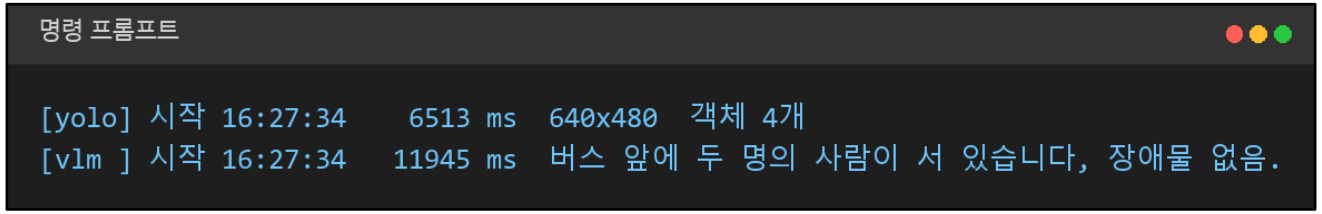

- LLM 칸 오류는 정상 - LLM 은 64쪽에서 붙인다
- 캡처 : 웹캠 대신 sample.jpg



Confidential

# 두 요청이 같은 시각에 시작해 겹쳐 돈다

- YOLO · VLM 이 같은 초에 시작
- 첫 요청은 모델 로딩 포함
- 전체 시간은 VLM (약 12초) 이 정한다
- 같은 사진 두 번째 요청 : YOLO 0.2s (66쪽 로그)



# 오류 대응 - 한 모델이 실패해도 나머지 결과는 보여 준다

Confidential

## YOLO 와 VLM 을 따로 요청하므로, 한쪽 오류가 다른 쪽 결과를 막지 않는다

- **YOLO 가 실패해도 VLM 요청은 따로 성공한다**
  - 화면은 되는 것만 채우고, 실패한 칸에만 'YOLO 인식 오류' 를 띄운다
- **실패는 예외가 아니라 약속된 응답으로**
  - 빈 본문 → 400 empty_image, 이미지 아님 → 400 invalid_image, 추론 실패 → 500 yolo_failed
  - 서버 트레이스백이 사용자 화면으로 가지 않는다
- **'실패' 와 '없음' 을 구분한다**
  - 객체가 없으면 200 + 빈 배열 (정상), 모델 파일이 없으면 500 (고장)
- **로그에 시작 시각과 소요 시간을 남긴다 - 어느 모델이 느린지 로그만 보고 안다**

# 도전 문제 3 - 서버 상태 확인 API

Confidential

## 대시보드_서버_도전3.py 의 health() 로 어떤 모델이 메모리에 올라와 있는지 보이게 하시오.

### 요구 사항

① 응답 예 : {"status": "ok", "yolo_loaded": false, ...}
② 서버를 켠 직후에는 모두 false
③ [전송] 한 번 뒤에는 모두 true
④ 확인만 한다 - 모델을 새로 올리지 않는다

### 힌트

· 모델은 _models 딕셔너리에 있다
· 아직 안 불렀으면 값이 None
· 지연 로딩 - 1일차에 배운 패턴
· 확인 : 브라우저 주소창에
127.0.0.1:8000/api/health
· 서버는 대시보드_서버_도전3.py 로 띄운다

전체 소스 : page49_대시보드_서버/대시보드_서버_도전3.py
https://gist.github.com/maketrust7/a631e5b32973dbff3aed3b74a68a0a42

Confidential

# _models 를 읽기만 하면 로딩 상태가 보인다

- None 이면 아직 안 올라온 것
- ④ _get() 을 부르면 로딩이 시작된다
- 실측 : YOLO 요청 뒤 yolo 만 True
- VLM 요청 뒤 둘 다 True

```python
@app.get("/api/health")
def health():
    return {"status": "ok", **{f"{name}_loaded": model is not None
                               for name, model in _models.items()}}
```

검증 – 정답 파일 실제 실행

```
[서버 시작 직후]  GET /api/health
  {'status': 'ok', 'yolo_loaded': False, 'vlm_loaded': False}

[YOLO 요청 1회 후 (3.5s)]  GET /api/health
  {'status': 'ok', 'yolo_loaded': True, 'vlm_loaded': False}

[VLM 요청 1회 후 (10.4s)]  GET /api/health
  {'status': 'ok', 'yolo_loaded': True, 'vlm_loaded': True}
```

# LLM 상황 통합

Confidential

# 보는 것은 인식 모델, 이해하는 것은 LLM

- **LLM(Qwen3-4B) 은 이미지를 보지 않는다**
  - 입력은 Text 세 줄 + 사용자 질문이 전부다
- **LLM 이 하는 일**
  - 세 출처를 종합해 상황을 설명한다 : '앞에 한 여성이 있고, 위를 가리키고 있다'
  - 출처끼리 다를 때 규칙에 따라 누구를 믿을지 정한다
- **LLM 이 하지 않는 일 (오늘)**
  - 개수 세기(YOLO) · 방향 계산(제스처) - 이미 인식 모델이 했다
  - 도구 호출로 로봇 움직이기 - 3일차

Confidential

# llama_cpp 하나로 VLM 과 LLM 을 모두 돌린다

- **Llama(model_path=...) 한 줄 - VLM 과 달리 mmproj 가 없다 (이미지를 안 받으니까)**
  - mmproj : 이미지를 조각으로 나눠 언어 모델이 읽는 토큰으로 바꾸는 부분 - VLM 만 필요
  - 이미지가 작을수록 이미지 토큰이 줄어 빨라진다 (448px 로 줄인 이유)
- **채팅 템플릿은 GGUF 안에 - create_chat_completion(messages=[...]) 만 부른다**
  - system : 규칙 (매번 같음)  user : 인식 Text + 질문 (매번 바뀜)
- **Qwen3 는 답 앞에 <think> ... </think> 를 먼저 쓴다 - 길면 느리고, 잘리면 답이 사라진다**
  - 끄는 법 : user 메시지 끝에 /no_think. 그래도 빈 <think></think> 가 붙는다 (실측)
  - 그래서 정규식으로 한 번 더 지운다 - 안 지우면 사고 과정이 답으로 노출된다

Confidential

# LLM 이 받는 입력은 이것이 전부다

- **한 줄에 한 출처, [출처] 를 맨 앞에**
- **블록 이름 = 규칙의 주어 (글자까지)**
- **빈 블록 금지 - '손 없음' 처럼 이유를**
- **순서 고정 : YOLO → VLM → 손 → 질문**

LLM 입력 (user 메시지)

```
[YOLO 객체 인식] person 1개, tie 1개
[VLM 장면 설명] 여성이 손가락으로 1을 가리키고 있다, 장애물 없음.
[손 제스처] Pointing_Up, 가리키는 방향: 위

[사용자 질문] 지금 앞에 무엇이 있고, 내가 어느 쪽을 가리키고 있어?
```

56쪽 실습 4-1 실측 예시

Confidential

# 출처마다 잘하는 것이 다르다 - 1일차의 역할 분담을 규칙으로 쓴다

- **규칙이 없으면 LLM 은 더 그럴듯한 문장 쪽을 따른다**

| 질문의 종류 | 따를 블록 | 이유 |
|---|---|---|
| 몇 개 / 몇 명 | [YOLO 객체 인식] | 개수는 검출 모델이 정확하다 (VLM 은 개수에 약하다) |
| 분위기 · 관계 · 상황 | [VLM 장면 설명] | 장면 전체를 서술할 수 있는 건 VLM 뿐이다 |
| 어느 쪽 / 방향 | [손 제스처] | 사용자의 의도는 손이 가장 직접적이다 |
| 블록에 없는 것 | (없음) | 지어내지 말고 '알 수 없다' |

Confidential

# system 에는 늘 같은 답변 규칙, user 에는 이번 인식 결과 3줄과 질문

- **system : 답변 규칙 5개 (항상 같음)**
- **user : 인식 결과 3줄 + 질문 (매번 바뀜)**
- **인식 결과 : 이 폴더의 perception.json**
- **실행 : python LLM_통합답변.py**

### 옵션 - 파일 이름 뒤에 붙인다

python LLM_통합답변.py --question "사람이 몇 명이야?"
python LLM_통합답변.py --yolo "person 2개"
python LLM_통합답변.py --vlm "두 사람이 마주 보고 있다"
→ 지정한 값만 바뀌고 나머지는 perception.json 그대로

```python
SYSTEM_PROMPT = (
    "너는 카메라 장면을 설명하는 로봇 어시스턴트다. 이미지를 직접 볼 수 없고, "
    "인식 모델이 만든 Text 블록만 근거로 답한다.\n"
    "규칙:\n"
    "1. 물체의 종류와 개수는 [YOLO 객체 인식] 을 따른다.\n"
    "2. 장면의 분위기·관계·상황은 [VLM 장면 설명] 을 참고한다.\n"
    "3. 사용자가 가리키는 방향은 [손 제스처] 를 따른다.\n"
    "4. 블록에 없는 내용은 지어내지 말고 '알 수 없다' 고 말한다.\n"
    "5. 한국어로 두세 문장으로 답한다."

# ...

def build_user_content(perception: dict, question: str) -> str:
    return (
        f"[YOLO 객체 인식] {perception['yolo']}\n"
        f"[VLM 장면 설명] {perception['vlm']}\n"
        f"[손 제스처] {perception['gesture']}\n\n"
        f"[사용자 질문] {question}"

# ...

            {"role": "user", "content": user_content + " /no_think"},
```

전체 소스 : page56_LLM_통합답변/LLM_통합답변.py
https://gist.github.com/maketrust7/649f6e1fa4819a458c833513fc1acb09

Confidential

# 세 줄의 Text 만으로 장면과 의도를 함께 설명한다

- **방향(위)은 [손 제스처] 를 따랐다**
- **사람은 VLM 문장대로 '여성'**
- **tie(오탐) 는 답에서 빠졌다**
- **추론 4.9s (CPU)**

```
명령 프롬프트

입력 출처 : perception.json

[LLM 입력]
[YOLO 객체 인식] person 1개, tie 1개
[VLM 장면 설명] 여성이 손가락으로 1을 가리키고 있다, 장애물 없음.
[손 제스처] Pointing_Up, 가리키는 방향: 위

[사용자 질문] 지금 앞에 무엇이 있고, 내가 어느 쪽을 가리키고 있어?

모델 로딩 중...
모델 로딩 : 2.4s

[LLM 답변]
앞에 여성의 손가락이 위를 가리키고 있습니다. 여성은 위쪽을 가리키고 있습니다.

추론 시간 : 4.9s  (입력 267 토큰 / 출력 32 토큰)
```

Confidential

# 3일차에는 LLM 의 답으로 tool 을 호출해 시뮬레이터를 움직인다

- **사람이 읽는 답 - 문장**
  - '위를 가리키고 있습니다' → 코드가 방향을 뽑아내려면 또 해석해야 한다
- **코드가 읽는 답 - JSON**
  - {"direction": "up", "obstacle": false, ...} → 바로 if 문에 넣는다
- **필드와 값의 범위를 프롬프트에 못 박는다**
  - direction 은 up / down / left / right / none 다섯 개 중 하나
  - 3일차 : 같은 파서 구조로 {"tool": "move", "args": {"direction": "up", "steps": 3}} 을 받는다

Confidential

# 4B 모델은 지정한 출력 형식을 자주 어긴다 - 전부 실제로 겪은 것들

- **json.loads() 한 줄로 받으면 서비스가 멈추거나(파싱 실패), 틀린 값으로 동작한다(파싱 성공)**

| 실패 유형 | 예 | 그대로 두면 |
|---|---|---|
| 앞뒤 잡글 / 코드펜스 | 네, 정리하면 {...} 참고하세요. | json.loads 실패 |
| 사고 블록 | <think></think>{...} | json.loads 실패 |
| 문자열 bool | "obstacle": "false" | bool("false") = True — 정반대 |
| 범위 밖 값 | "direction": "오른쪽" | if direction == "right" 가 안 걸림 |
| 형식 변형 | "objects": "person, tie" | YOLO Text 규격(person 1개)과 다름 (실측) |

Confidential

# 모델이 무엇을 내놓든 항상 같은 모양의 dict 를 돌려준다

- **① <think> 블록 제거**
  - 닫히지 않고 잘린 블록까지 정규식으로
- **② 첫 '{' ~ 마지막 '}' 만 잘라 파싱**
  - 앞뒤 잡글과 코드펜스가 한 번에 사라진다
- **③ 실패해도 형식은 유지**
  - ok=False 와 기본값을 채워 돌려준다 - 호출하는 쪽이 예외 처리를 몰라도 된다
- **④ 필드별 타입 보정**
  - to_bool('false') → False,  to_direction('오른쪽') → 'right'

Confidential

# 형식은 모델이 아니라 파서가 보장한다

- [A] LLM 실제 호출 → 파싱
- [B] 깨진 출력 6종으로 파서 점검
- 실행 ① 전체([A]+[B]) : python LLM_JSON파싱.py
- 실행 ② 파서만([B]) : python LLM_JSON파싱.py --skip-llm
- 둘 중 하나만 실행 - 모델 파일이 없으면 ② (몇 초)

```python
def to_bool(value) -> bool:
    if isinstance(value, str):
        return value.strip().lower() not in ("", "false", "0", "no", "none", "null")
    return bool(value)

# ...

def parse_llm_output(raw: str) -> dict:
    text = _THINK_RE.sub("", raw).strip()           # 1) 사고 블록 제거
    data = extract_json(text)                        # 2) JSON 구간만 추출
    if data is None:                                 # 3) 실패해도 형식은 유지
        return {"ok": False, "objects": "", "obstacle": False,
                "direction": "none", "summary": text or "답변 없음"}
    return {                                         # 4) 필드별로 타입 보정
        "ok": True,
        "objects": str(data.get("objects", "")).strip(),
        "obstacle": to_bool(data.get("obstacle", False)),
        "direction": to_direction(data.get("direction", "none")),
        "summary": str(data.get("summary", "")).strip(),
```

전체 소스 : page61_LLM_JSON파싱/LLM_JSON파싱.py
https://gist.github.com/maketrust7/d7098008fb4b33149e7fce3f2f6e0113

Confidential

# 5종은 값을 되살리고, 잘린 1종은 ok=False 와 기본값(멈춤)을 돌려준다

- **원문에 빈 <think></think> 가 실제로 붙어 나왔다**
- **잘린 think → ok=False + 기본값**
- **direction=none 이면 로봇이 멈춘다 - 잘못 움직이는 것보다 낫다**
- **bool("false") 는 True - 파서가 막는다**

| 유형 | ok | 파서가 돌려주는 값 |
|---|---|---|
| 정상 · 코드펜스 · 앞뒤 잡글 · 문자열 bool · 한글 방향 (5종) | True | 원래 값을 되살림 |
| 잘린 think (1종) | False | 기본값 : obstacle=False, direction=none, summary='답변 없음' |

```
명령 프롬프트

[A] LLM 실제 호출
  입력 :
    [YOLO 객체 인식] person 1개, tie 1개
    [VLM 장면 설명] 여성이 손가락으로 1을 가리키고 있다, 장애물 없음.
    [손 제스처] Pointing_Up, 가리키는 방향: 위

    [사용자 질문] 지금 상황을 정리해줘.

  원문 (5.8s) :
    <think>

    </think>

    {"objects": "person, tie", "obstacle": false,
    "direction": "up", "summary": "여자가 손가락으로 위를 가리키며 1을
    가리키고 있다."}

  파싱 결과 :
    ok        : True
    objects   : person, tie
    obstacle  : False
    direction : up
    summary   : 여자가 손가락으로 위를 가리키며 1을 가리키고 있다.

[B] 파서 자체 점검 — 깨진 출력 6종
  유형         ok     obstacle  direction summary
  정상         True   False     up        한 사람이 위를 가리킨다.
  코드펜스      True   False     up         위를 가리킨다.
  앞뒤 잡글     True   True      none       버스가 길을 막고 있다.
  문자열 bool  True   False     left      왼쪽을 가리킨다.
  한글 방향     True   False     right      오른쪽을 가리킨다.
  잘린 think  False  False     none      답변 없음

  참고: 파서 없이 bool("false") 로 읽으면 obstacle = True  ← 정반대가 된다

형식은 모델이 아니라 파서가 보장한다 → 3일차 도구 호출의 입력이 된다.
```

Confidential

# 브라우저가 인식 결과와 질문을 모아 보내면, 서버는 LLM에 입력으로 넣는다

- **입력은 브라우저가 보낸다**
  - YOLO · VLM 결과는 앞 요청에서 서버가 돌려준 것을 브라우저가 다시 보낸다
  - 질문 + YOLO 요약(앞에서 정한 규격) + VLM 문장 + 손 방향(브라우저 제스처)
- **블록과 규칙은 실습 4-1 그대로**
  - [손 제스처] 는 방향만 온다 - '가리키는 방향: 위' 또는 '인식 불가'
- **응답은 {need_more_info, message, user_content}**
  - user_content : LLM 에 실제로 넣은 입력. 브라우저가 대화 기록으로 보관한다
  - 오늘은 need_more_info 가 항상 false - 되묻기는 3일차
- **오늘은 설명까지만 - 시뮬레이터 로봇은 움직이지 않는다**
  - 3일차 : LLM 이 질문에 맞는 tool 을 고르고, 시뮬레이터 상의 로봇이 움직인다

Confidential

# API 하나를 더하면 답변창에 LLM 답변이 적힌다

- **실습 3-1 서버 + /api/llm-answer**
- **서버 창에 LLM 입력이 그대로 찍힌다**
- **실행 : python 대시보드_LLM연결.py**
- **접속 : http://127.0.0.1:8000**
- **확인 : 로컬 PC 탭에서 '사람이 몇 명이야?' 입력 → [전송]**

```python
def build_user_content(prompt: str, yolo_summary: str, vlm_description: str,
                       gesture_direction: str) -> str:
    direction = DIRECTION_KO.get(gesture_direction, "인식 불가")
    return (
        f"[YOLO 객체 인식] {yolo_summary.strip() or '없음'}\n"
        f"[VLM 장면 설명] {vlm_description.strip() or '없음'}\n"
        f"[손 제스처] 가리키는 방향: {direction}\n\n"
        f"[사용자 질문] {prompt}"

# ...

    user_content = build_user_content(
        prompt,
        str(body.get("yolo_summary", "")),
        str(body.get("vlm_description", "")),
        str(body.get("gesture_direction", "unknown")))

# ...

        message = await run_in_threadpool(answer, user_content)

# ...

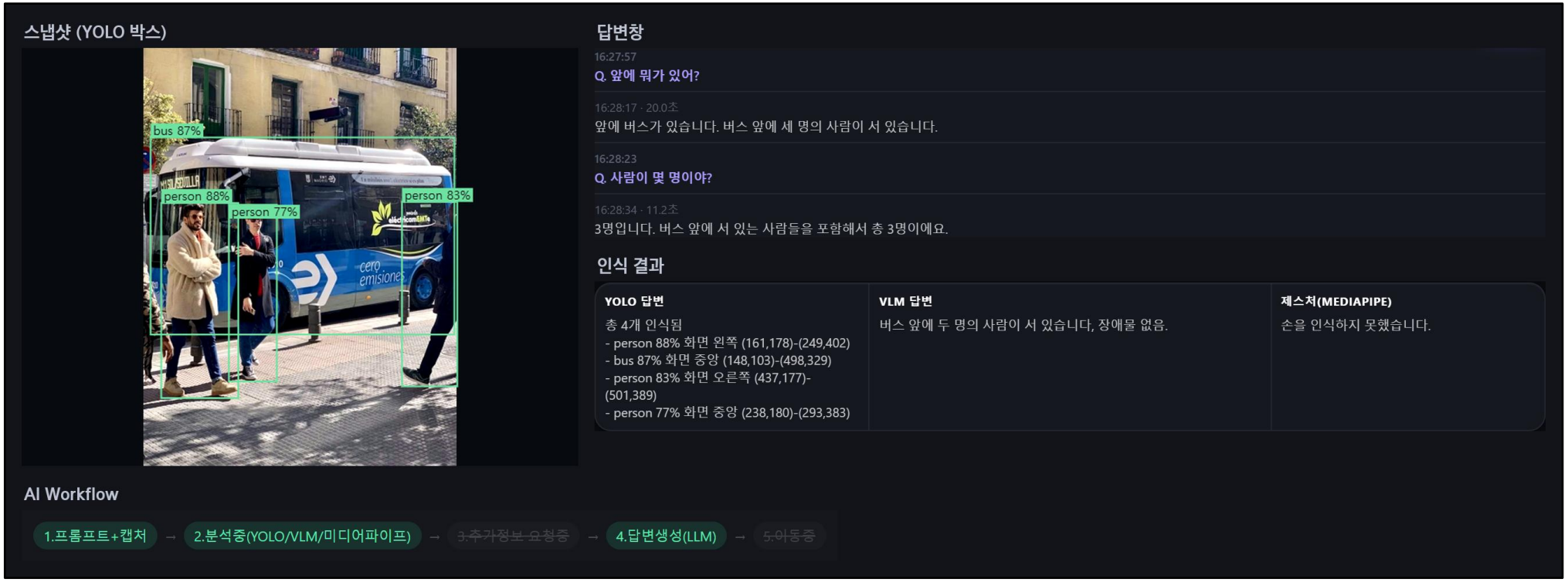
    return {"need_more_info": False,
            "message": message,
            "user_content": user_content}
```

전체 소스 : page64_대시보드_LLM연결/대시보드_LLM연결.py
https://gist.github.com/maketrust7/9a617f377a9a669713693ffaa9383b78

Confidential

# 사용자 질문에 YOLO · VLM · 손 제스처 결과를 묶어 LLM 이 답한다

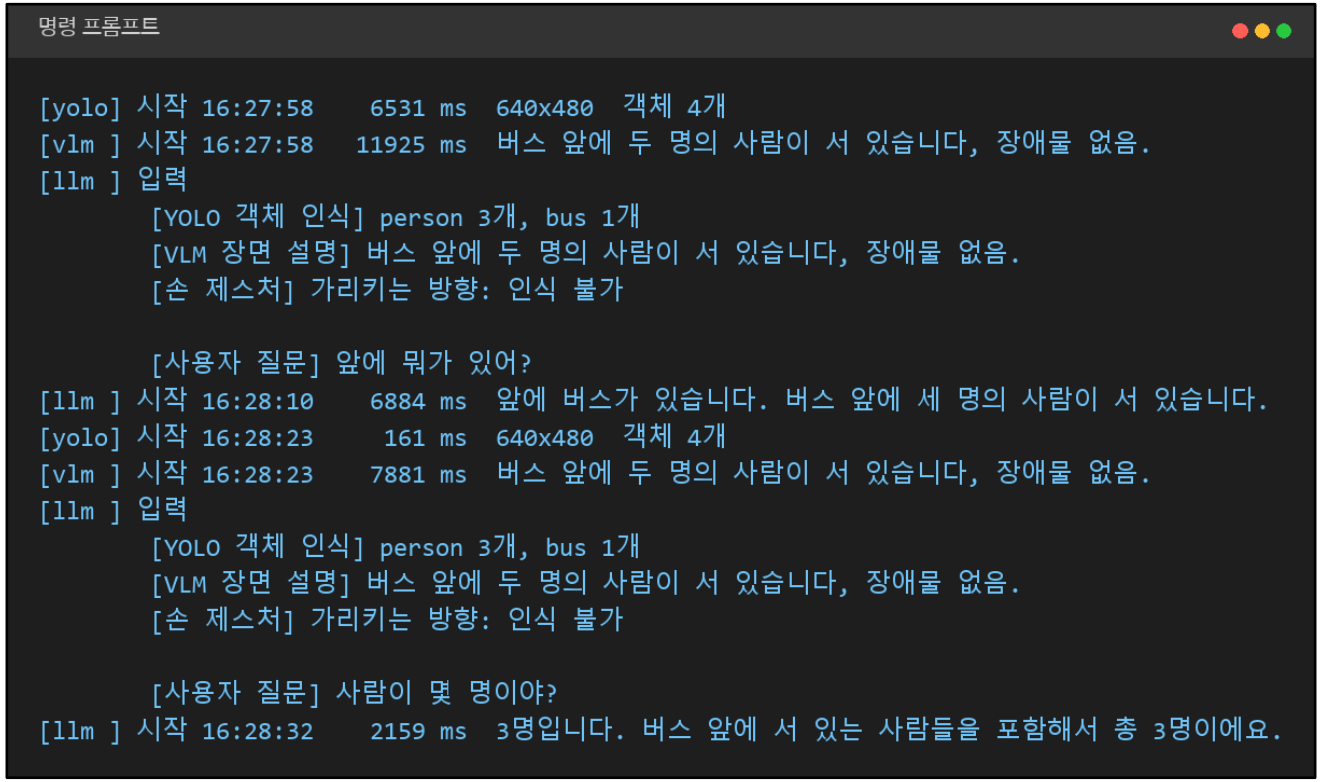
- **Q. 사람이 몇 명이야? → 3명**
- **YOLO 3명 vs VLM 2명 → 개수는 YOLO 를 따랐다**
- **추가정보 · 이동 단계는 건너뜀**
- **캡처 : 웹캠 대신 sample.jpg**



Confidential

# LLM 이 받은 입력은 이 다섯 줄이 전부다

- **브라우저가 보낸 값으로 블록을 만든다**
- **손이 없으면 '인식 불가'**
- **두 번째 YOLO 0.2s (모델 상주)**
- **LLM 2.2 ~ 6.9 s (CPU)**



# 도전 문제 4 - 충돌을 숨기지 않는 LLM

Confidential

## LLM_통합답변_도전4.py 에 규칙 한 줄(RULE_6)을 추가해, 출처끼리 다를 때 그 사실까지 알리게 하시오.

### 지금의 답 (실측)

[YOLO 객체 인식] person 4개, bus 1개
[VLM 장면 설명] 버스 앞에 남성이 서 있다,
장애물 없음.
[손 제스처] 손 없음
[사용자 질문] 사람이 몇 명이야?
→ 4명입니다. 버스 앞에 서 있는 남성들입니다.
(VLM 원문 '버스 앞에 남성이 서 있다' 는 답변에서 생략되었다)

### 요구 사항과 힌트

① 개수는 여전히 YOLO 를 따른다
② VLM 이 어떻게 말했는지 덧붙인다
③ VLM 의 말을 지어내면 안 된다
· 규칙 추가 : RULE_6 = "..."
· 두 번 실행해 같은 답인지 확인

전체 소스 : page67_LLM_통합답변/LLM_통합답변_도전4.py
https://gist.github.com/maketrust7/ce22bb7382383606b5c8da8de61ece3e

# 도전 4 해설 - '다르면' 대신 '항상' 과 '원문 인용'

Confidential

## 4B 모델에게 '다른지 판단' 을 맡기지 말고, 할 일을 조건 없이 적는다

- **시도 1 '비교해서 다르면 알려라' → 무시하거나, 결론이 1명으로 뒤집혔다 (실측)**
- **시도 2 '다르면 덧붙여라' → LLM 이 'VLM 은 4명이라고 설명했다' 고 답함 - VLM 원문에 없는 말을 지어냈다 (실측)**
- **정답 : '개수는 끝까지 YOLO' + 'VLM 원문을 그대로 인용' → 새로 실행해도 같은 답**

```
정답 파일 실행 (실측, 규칙과 답변만)

추가 규칙 : 6. 개수는 끝까지 [YOLO 객체 인식] 을 따른다. 단, [VLM
             장면 설명] 의 원문을 그대로 인용해 VLM 은 어떻게
             설명했는지 한 문장으로 덧붙인다.


[LLM 답변]  (6.4s)
4명입니다. YOLO 객체 인식에 따르면 사람은 4명이 있습니다. "버스 앞에 남성이
서 있다"고 설명했습니다.

[자동 확인]
  O  ① 개수는 YOLO (4명)
  O  ② VLM 원문 인용 ('버스 앞에 남성이 서 있다')
  ?  ③ 지어낸 말이 없는지는 직접 읽고 판단한다
```

Confidential

# 실습 중 막히면 여기부터 확인한다

| 증상 | 원인 | 해결 |
|---|---|---|
| ggml-cuda ... CUDA error 로 서버 종료 | torch(GPU) 와 llama.cpp(GPU) 가 한 프로세스에서 충돌 | CUDA_VISIBLE_DEVICES=-1 로 CPU 고정, 또는 한쪽만 GPU |
| 대시보드 화면이 안 뜸 / 웹캠이 안 켜짐 | html 파일을 직접 엶 / 카메라 권한 거부 | 서버 실행 후 http://127.0.0.1:8000 으로 접속, 권한 허용 |
| YOLO · VLM 칸에 '오류' | 서버에서 추론 실패 | 서버 창의 [yolo] / [vlm] 로그 확인 |
| 답변에 <think> 가 보임 | 사고 블록 제거 누락 | /no_think + 정규식 제거 |
| JSONDecodeError | JSON 앞뒤 잡글 · 코드펜스 | 첫 '{' ~ 마지막 '}' 만 파싱 |
| LLM 이 개수를 VLM 에서 가져옴 | 출처 규칙 없음 / 블록 이름 불일치 | 규칙과 블록 이름을 글자까지 맞춘다 |
| 답에 영어 · 한자가 섞임 ('사람 three 명') | 4B · 3B 소형 모델의 한계 (PC 마다 다름) | 같은 질문 한 번 더. 규칙에 '한국어로만' 을 둔다 |
| 첫 요청만 오래 걸림 | 모델 지연 로딩 | 정상. 두 번째부터 빨라진다 |

Confidential

# 내용 정리

- ① LLM 이 보는 세상은 Text 뿐 - 인식 결과의 형식이 곧 입력 설계다
- ② 형식은 프롬프트/max_tokens/코드 세 군데에 건다
- ③ 제스처의 방향은 카테고리가 아니라 랜드마크 벡터로 구한다
- ④ 화면은 주어지고 서버는 API 로 - VLM 은 '읽는 시간' 을 줄여서 쓴다
- ⑤ 출처별 블록 + 누구를 믿을지 규칙 - 개수는 YOLO, 상황은 VLM, 방향은 손
- ⑥ 형식은 모델이 아니라 파서가 보장한다 - 3일차 도구 호출의 전제

Confidential

# 필수 및 선택 기능 구현

- **① (필수) 내 손으로 찍은 사진 3장을 실습 2-2 에 넣어 본다**
  - 셀카 카메라라면 --mirror 가 필요한지 직접 확인한다 - 참고 : 31~32쪽 (실습 2-2)
- **② (필수) 도전 2 의 count_fingers() 를 실습 2-2 의 Text 에 붙인다**
  - 형식 예 : Victory, 가리키는 방향: 위, 손가락 2개 - 참고 : 36~37쪽 (도전 2)
- **③ (선택) 과제_VLM_단축측정 으로 내 PC의 vlm_speed.csv 를 남긴다**
  - A · B · C 세 설정의 시간을 잰다. 대시보드는 설정 C 로 돈다 - 참고 : 40~41쪽
- **④ (선택) 대시보드에서 질문 3개를 던지고 답을 기록한다**
  - 개수 · 상황 · 방향 질문을 하나씩, 틀린 답은 부족한 규칙을 적는다 - 참고 : 64~65쪽 (실습 4-3)

Confidential

# LLM 이 인식 결과 Text 를 보고 도구(tool)를 고르고, JSON 으로 답한다

- **의도 파악 - 사용자가 원하는 것이 설명인가, 이동인가**
  - '앞에 뭐 있어?' 는 설명, '오른쪽으로 3칸 가' 는 이동
- **도구 호출(tool call) - LLM 이 호출할 도구와 인자를 JSON 으로 고른다**
  - 예 : move(direction="up", steps=3), describe(), ask()
  - 오늘의 방어 파서가 그 JSON 을 받는다
- **조건 분기 - 정보가 부족하면 되묻는다**
  - '저쪽으로 가' 처럼 방향을 모르면 움직이지 않고 '어느 쪽으로 갈까요?' 로 되묻는다
- **시뮬레이터 - LLM 이 move 를 고르면 대시보드의 격자 맵 로봇이 움직인다**
  - 오늘 만든 대시보드 서버의 /api/llm-answer 에 tool 을 더한다
- **준비물 : 실습 4-3 서버가 뜨는 상태 (웹캠은 제공)**

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Q & A

Confidential

# 모든 명령은 C:₩work₩core_pjt₩source₩2회차_day2 에서 실행한다

- 옵션은 -h 로 확인할 수 있다
- 생성물은 각 실습 폴더에 생긴다
- 대시보드는 서버를 띄운 뒤 브라우저로
- 화면 파일은 ..₩2회차_web (공용)

```
# 실습 한 건 = 폴더 한 개. 폴더 앞 숫자 = 이 강의자료 쪽번호
cd C:\work\core_pjt\source\2회차_day2

# 도입 — 환경 확인
cd page10_환경체크              && python 환경체크_설치확인.py

# Part 1 — 인식 결과를 텍스트로
cd ..\page14_YOLO_텍스트요약    && python YOLO결과_텍스트요약.py
cd ..\page18_VLM_출력규격       && python VLM_출력규격.py

# Part 2 — MediaPipe 제스처
cd ..\page27_제스처_인식        && python 제스처_인식.py --save
cd ..\page31_제스처_방향판별    && python 제스처_방향판별.py

# Part 3 — 웹 대시보드 서버 (띄운 뒤 브라우저로 접속)
cd ..\page45_대시보드_서버      && python 대시보드_서버.py
#   → http://127.0.0.1:8000

# Part 4 — LLM 상황 통합
cd ..\page56_LLM_통합답변       && python LLM_통합답변.py
cd ..\page61_LLM_JSON파싱       && python LLM_JSON파싱.py
cd ..\page64_대시보드_LLM연결   && python 대시보드_LLM연결.py

# 기능 구현 ③ — 내 PC의 VLM 단축 효과
cd ..\과제_VLM_단축측정         && python VLM_단축측정.py
```

Confidential

# 실습 중 가장 많이 나오는 질문

- **먼저 이 표에서 찾아본 뒤 질문해 주세요**

| 증상 / 에러 메시지 | 원인 | 해결 |
|---|---|---|
| ModuleNotFoundError: mediapipe | .venv 가 아닌 인터프리터 / 미설치 | .venv 활성화 후 pip install mediapipe==1.0.1 |
| 제스처 모델 파일이 없습니다 | gesture_recognizer.task 미다운로드 | models₩ 에 배치 (설치 ① 의 curl) |
| LLM 파일이 없습니다 | Qwen3-4B GGUF 미다운로드 | hf download (설치 ① 의 마지막 줄) |
| load_hparams: Qwen-VL models require ... | llama.cpp 안내 로그 | 오류가 아님. 무시 |
| [Errno 10048] bind on address | 8000 포트를 이미 다른 서버가 사용 | 1일차 PoC 서버 종료 |
| CPU 사용률 100% 로 PC 가 느려짐 | VLM · LLM 이 모든 코어를 사용 | 정상. 무거운 실습은 하나씩 실행 |

Confidential

# 브라우저와 대시보드 서버가 주고받는 형식 - 3일차 API 설계의 A1 ~ A5

- **page45 대시보드 서버 = A1 ~ A4, page64 LLM 연결 = A5 추가 - 오류는 모두 {"error": "코드"}**

| 번호 | 메서드 · 경로 | 요청 | 응답 (200) | 오류 |
|---|---|---|---|---|
| A1 | GET / | — | index.html | — |
| A2 | GET /mediapipe/... | — | 제스처 인식 파일 (vision_bundle.mjs · wasm · .task) | 404 |
| A3 | POST /api/yolo-detect | JPEG 바이트 | {objects: [{label, confidence, box}]} | 400 empty_image · invalid_image, 500 yolo_failed |
| A4 | POST /api/vlm-describe | JPEG 바이트 | {description: 문자열} | 400 empty_image · invalid_image, 500 vlm_failed |
| A5 | POST /api/llm-answer | JSON (아래 표) | {need_more_info: false, message, user_content} | 400 invalid_json · empty_prompt, 500 llm_failed |

| A5 요청 필드 | 필수 | 설명 |
|---|---|---|
| prompt | O | 사용자 질문. 공백뿐이면 400 empty_prompt |
| yolo_summary · vlm_description | | A3 · A4 결과 (브라우저가 요약해 보낸다) |
| gesture_direction | | 손 방향 up · down · left · right · unknown |
| (3일차) | | 응답에 tool · args 가 더해지고, 되묻기용 history 를 받는다 |

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 내일 방송에서 만나요!

삼성청년SW·AI아카데미